

Solution TD N°1

Exercice 1:

Supposer que deux utilisateurs visitent le site en même temps. Deux processus P1 et P2 seront créés. Ils correspondent tous les deux au programme «**CompteVisite**». Supposons que le compteur du nombre de visites soit égal à N et soit la séquence suivante :

- a) P1 exécute les étapes (1) à (3) puis est interrompu
- b) P2 exécute la séquence (1) à (9)
- c) P1 reprend son exécution et exécute les étapes (4) à (9)

Résultat : La valeur du compteur devient N+1 (au lieu de N+2).

Remarque :

D'autres séquences peuvent produire ce même résultat (incorrect)
Généraliser au cas où plusieurs utilisateurs accèdent en même temps.

Exercice 2:

L'idée est de décomposer chacune des instructions C ($x=x+1$, $x=x*2$) en plusieurs instructions assembleur, ceci met en évidence la possibilité d'interruption en cours d'exécution de chacune des instructions plus haut.

- a) `int x=0; // variable partagée déclarée dans un contexte global`

P1:	P2:
.....	
a) LDA x	a') LDA x
b) ADD #1	b') MUL #2
c) STA x	c') STA x
<code>printf ("x=%d\n", x) ;</code>	<code>printf ("x=%d\n", x) ;</code>
.....	

Considérer ensuite la séquence d'exécution suivante (par exemple):

(a, b, c, a', b', c') : P1 affiche **1** et P2 affiche **2**

(a, b, a', b', c', c) : P1 affiche **1** et P2 affiche **0**

Et ainsi de suite avec les autres séquences possibles

- b) Même idée en éclatant les 2 instructions qui manipulent x et y dans les 2 processus.

Exercice 3 :

1. Donner pour chaque fichier les sections critiques de PA, PB et PC.
F1 : pas de section critique car il n'est pas accédé en écriture
F2 : A2 et C4
F3 : A4, B2 et C1
2. En déduire les sections en exclusion mutuelle.
Sections en exclusion mutuelle :
A2 et C4 (F2)
A4, B2 et C1 (F3)

Exercice 4 :

1. Progression

Un processus qui n'a pas l'intention d'utiliser la SC ne doit pas interdire l'accès à un autre processus qui en a besoin.

Supposons que P0 n'a pas besoin de la SC, alors B0 = false car,

Si P0 n'est jamais entré en SC, B0 est initialisée à false,

Si P0 a quitté la SC, il a remis B0 à false dans le protocole de sortie

Supposons que le processus P1 veut accéder à la SC, alors B1 = true

- Si B0 est false, alors P1 passe en SC.

Donc, Si P0 ne veut pas accéder à la SC, P1 peut le faire sans problème.

2. Interblocage

Le scénario suivant montre une situation d'interblocage, ou chaque processus attend que l'autre lui cède la place

P0 : B0 = true

P1 : B1 = true

P0 : while(B1) vrai

P0 bloqué

P1 : while(B0) vrai

P1 bloqué

Exercice 5 :

Solution non valide, soit le scénario suivant (supposons que $turn == 0$)

Scenario 1:

1. P0 : $flag[0] = 1$
2. P0 : $while (turn != 0)$ (faux)
3. P0 : entre en SC
4. P0 : $flag[0] = 0$
5. P1 : $flag[1] = 1$
6. P1 : $while (turn != 1)$ (vrai)
7. P1 : $while (flag[0] == 1)$ (faux)
8. P0 : $flag[0] = 1$
9. P0 : $while (turn != 0)$ (faux)
10. P0 : entre en SC
11. P1 : $turn = 1$
12. P1 : $while (turn != 1)$ (faux)
13. P1 : entre en SC

Alors, les 2 processus entrent ensemble en SC, la solution ne garantit pas l'exclusion mutuelle, donc elle n'est pas valide.

Scenario 2: plus optimal

1. P1 : $flag[1] = 1$
2. P1 : $while (turn != 1)$ (vrai)
3. P1 : $while (flag[0])$ (faux)
4. P0 : $flag[0] = 1$
5. P0 : $while (turn != 0)$ (faux)
6. P0 : entre en SC
7. P1 : $turn = 1$
8. P1 : $while (turn != 1)$ (faux)
9. P1 : entre en SC

Exercice 6 :

Utilisation de l'instruction « exchange » La réalisation de l'exclusion mutuelle peut se faire comme suit :

```
const int N=... ; // nombre de processus en Exclusion Mutuelle
int verrou ;
```

```
void P (int i) // i représente le numéro du processus
{
    int cle ;
    while (1) {
        cle=1 ;
        while (cle!=0) exchange (&cle, &verrou) ; // protocole d'entrée

        Section critique

        exchange (&cle , &verrou) ;           // protocole de sortie
    }
}

int main ( ) {
    verrou = 0 ;
    parbegin (P(0), P(1), ..... , P(N)) ; // Appel parallèle de fonctions
    return 0 ;
}
```

Le plus important dans cet exercice est que l'instruction « exchange » (comme « TestAndSet ») est une instruction « atomique ». Elle fait partie des jeux d'instructions du processeur ; donc elle s'exécute sans possibilité d'interruption.

De plus, puisque l'accès à la même adresse mémoire (la variable « verrou ») à un moment donné par plusieurs processus est impossible (caractéristique de la mémoire). La solution marche bien sur les systèmes monoprocesseurs ou multiprocesseurs. Et comme indiqué dans l'exercice, la solution est facilement généralisable à un nombre quelconque de processus.